

MCP Client & Server Authentication, Authorization & RBAC

A beginner-friendly guide to every concept
used in building secure MCP backends

Covers: MCP protocol, OAuth 2.1, JWT, API keys, RBAC,
industry RFC standards, and how they fit together.

Orchestrator Platform - Backend Concepts

Part 1: MCP Basics

What is MCP (Model Context Protocol)?

MCP is an open standard that lets AI applications (like chatbots and agents) connect to external tools and data sources in a consistent way. Think of it as a universal plug that lets an AI assistant use Gmail, GitHub, databases, or your own backend APIs without custom integration code for each one.

Analogy: MCP is like USB-C for AI tools. Before USB-C, every device had a different charger. MCP gives AI apps one standard way to discover and use tools.

MCP Client vs MCP Server

In every MCP interaction there are two sides:

- MCP Client - The AI application that WANTS to use tools. It asks: 'What tools do you have?' and 'Please run this tool.'
- MCP Server - The system that PROVIDES tools. It answers: 'Here are my tools' and executes them when asked.

Example: In your Orchestrator platform

OUTBOUND: Orchestrator is the MCP Client. It connects to external MCP servers (e.g. GitHub MCP) to use their tools.
INBOUND: Orchestrator can also BE an MCP Server, exposing its own agents and workflows as tools to external clients like Cursor.

What is a Tool in MCP?

A tool is a callable function exposed by an MCP server. Each tool has a name, a description (so the AI knows when to use it), and an input schema (what arguments it accepts). The AI reads tool descriptions and decides which tool to call.

Example: GitHub MCP tool

Tool name: create_issue
Description: Create a new GitHub issue
Input: { repo: string, title: string, body: string }

JSON-RPC - How MCP Messages Work

MCP uses JSON-RPC 2.0, a simple message format for remote procedure calls. Every request is a JSON object with: jsonrpc (always '2.0'), id (request number), method (what to do), and params (arguments). The server responds with result or error.

Example: tools/list request

```
{ "jsonrpc": "2.0", "id": 1, "method": "tools/list", "params": {} }
```

Example: tools/call request

```
{ "jsonrpc": "2.0", "id": 2, "method": "tools/call", "params": { "name": "search", "arguments": { "query": "hello" } } }
```

HTTP Transport (Streamable HTTP)

Remote MCP servers communicate over HTTP. The client sends JSON-RPC messages via POST requests. 'Streamable HTTP' is the modern transport that supports streaming responses for long-running operations. This is how SaaS MCP servers work over the internet.

- POST /mcp - Main endpoint for JSON-RPC messages
- Content-Type: application/json
- Authorization: Bearer <token> - When auth is required

Part 2: Authentication vs Authorization

Authentication (AuthN) - WHO are you?

Authentication verifies identity. It answers: 'Are you really who you claim to be?' Common methods: username/password login, API keys, OAuth tokens, JWT tokens.

Analogy: Authentication is showing your ID card at a building entrance. The guard checks that the ID is real and belongs to you.

Authorization (AuthZ) - WHAT can you do?

Authorization happens AFTER authentication. It checks whether an authenticated user is allowed to perform a specific action on a specific resource.

Analogy: Authorization is your employee badge determining which floors you can access. Your ID proves who you are; your badge level decides what you can do.

Concept	Question	Example
Authentication	Who are you?	Login with email/password
Authorization	What can you do?	Can you delete this agent?
RBAC	What is your role?	Builder can edit, Viewer cannot

Part 3: Authentication Methods

JWT (JSON Web Token)

A JWT is a signed string that contains user information (claims) like user ID, organization ID, and role. After login, the server gives the client a JWT. The client sends it in every request: Authorization: Bearer <jwt>. The server verifies the signature - no database lookup needed for basic checks.

- Signed with a secret key - tampering is detected
- Contains expiry (exp) - tokens expire automatically
- Used for dashboard users in Orchestrator

Example: JWT payload (decoded)

```
{ "sub": "user-uuid", "org_id": "org-uuid", "role": "builder", "exp": 1724000000 }
```

API Keys

API keys are long random strings for machine-to-machine access. Unlike JWTs from login, API keys are created by admins and given to scripts, workers, or external services. They are stored as hashes in the database (never plain text).

- Format in Orchestrator: oak_<prefix>_<secret>
- Can have scopes (permissions) and resource limits
- Can have rate limits and expiry dates

Analogy: API keys are like a hotel key card given to a cleaning robot. It works 24/7 without a human logging in, but only opens specific doors.

Bearer Tokens

A Bearer token is any secret sent in the HTTP Authorization header as 'Authorization: Bearer <token>'. Both JWTs and OAuth access tokens use this format. Whoever bears (holds) the token gets access - so keep it secret!

OAuth 2.1 - Industry Standard for MCP

OAuth 2.1 is the authorization framework required by the MCP specification for remote (HTTP) servers. It lets a user grant an application access to a service WITHOUT sharing their password. The user logs in at the real service (e.g. Google), approves access, and the app gets a limited access token.

Key OAuth Roles

- Resource Owner - The human user who owns the data
- Client - The app wanting access (e.g. Orchestrator MCP client)
- Authorization Server (AS) - Issues tokens after login (e.g. Keycloak, Auth0)
- Resource Server (RS) - The API/MCP server that accepts tokens (e.g. GitHub MCP)

Authorization Code Flow (with PKCE)

The standard flow for MCP clients connecting to remote servers:

- 1. Client discovers auth server URL from MCP server metadata
- 2. Client redirects user to login page (Authorization Server)
- 3. User logs in and approves requested permissions (scopes)
- 4. Auth server redirects back with a short-lived authorization code
- 5. Client exchanges code for access token (+ refresh token)
- 6. Client calls MCP server with: Authorization: Bearer <access_token>

PKCE (Proof Key for Code Exchange)

PKCE (pronounced 'pixie') prevents attackers from stealing authorization codes. Before redirecting the user to login, the client generates a random 'code_verifier' and sends a hashed version ('code_challenge') to the auth server. When exchanging the code for a token, the client must prove it has the original verifier. PKCE is MANDATORY in MCP OAuth flows.

Analogy: PKCE is like a two-part ticket. You get half at the start and must show both halves to collect your prize. An attacker who steals only one half cannot win.

Refresh Tokens

Access tokens expire quickly (e.g. 1 hour) for security. A refresh token is a longer-lived secret used to get new access tokens without making the user log in again. The MCP client should automatically refresh before calling tools.

Client Credentials Grant (Machine-to-Machine)

For server-to-server communication (no human user), the client authenticates directly with client_id + client_secret to get a token. Useful for background workers calling MCP servers. Support in MCP spec is evolving.

Part 4: Industry RFC Standards for MCP

RFC 9728 - Protected Resource Metadata

Every MCP server MUST expose a discovery document at:

`/.well-known/oauth-protected-resource`

This JSON file tells clients: 'To access me, go to these authorization servers.' Clients read this BEFORE starting OAuth, so they know where to send users to login.

Example: Protected Resource Metadata

```
{ "resource": "https://mcp.example.com", "authorization_servers": ["https://auth.example.com"] }
```

RFC 8414 - Authorization Server Metadata

The authorization server exposes its own discovery document with endpoints like `authorization_endpoint` (login page URL), `token_endpoint` (exchange code for token), and supported grant types. Clients use this to know exactly which URLs to call.

RFC 8707 - Resource Indicators (Critical Security)

When requesting a token, the client MUST include a 'resource' parameter set to the MCP server URL. This binds the token to ONE specific server. Without this, a malicious fake MCP server could steal tokens meant for the real server.

Analogy: RFC 8707 is like writing the delivery address on a package. The token can only be delivered to (used at) that exact address, not a lookalike address.

RFC 7591 - Dynamic Client Registration

Optional standard allowing MCP clients to automatically register themselves with an authorization server (get a `client_id`) without manual admin setup. Helpful for SaaS platforms connecting to many different MCP servers.

RFC	Name	Purpose
RFC 9728	Protected Resource Metadata	Find auth server from MCP URL
RFC 8414	AS Metadata	Find OAuth endpoints
RFC 8707	Resource Indicators	Bind token to one server
RFC 7591	Dynamic Registration	Auto-register OAuth client

Part 5: RBAC (Role-Based Access Control)

What is RBAC?

RBAC assigns permissions to ROLES, and roles to USERS. Instead of giving each user individual permissions, you give them a role (like 'Admin' or 'Viewer') and the role carries a fixed set of permissions. This is easier to manage in teams.

Analogy: RBAC is like job titles in a company. A 'Manager' can approve expenses; an 'Intern' cannot. You assign a title, not 50 individual permissions per person.

Roles in Orchestrator

Role	Level	Typical use
owner	Highest	Org creator, full control
admin	High	Manage users and settings
builder	Medium	Create agents and workflows
viewer	Low	Read-only access

Permissions (Fine-grained actions)

Permissions are strings like 'mcp:read' or 'agent:invoke'. Code checks: does this user's role include this permission? The `has_permission(role, permission)` function in `rbac.py` does this lookup.

- `mcp:read` - View MCP connections and discovered tools
- `mcp:write` - Create, update, delete MCP connections
- `mcp:invoke` - Allow agents to call MCP tools at runtime
- `agent:invoke` - Run an agent
- `api_key:write` - Create API keys

Scopes (for API Keys)

Scopes are the API key equivalent of permissions. When creating an API key, you choose which scopes it has. A key with only 'agent:invoke' cannot manage MCP connections even if the creating user is an admin.

Multi-Tenancy (Organization Isolation)

In a SaaS platform, every resource belongs to an organization (`org_id`). Even if User A is authenticated, they can NEVER access Organization B's agents, MCP connections, or runs. Authentication proves identity; `org_id` scoping ensures tenant isolation.

Part 6: Tool Policies & Security Layers

Tool Allowlist

An allowlist is a list of tool names that ARE permitted. If allowlist is empty, all discovered tools may be used (depending on policy). If allowlist has entries, only those tools can be called. This prevents an agent from accidentally using dangerous tools like 'delete_database'.

5-Layer Security Model (Recommended)

Industry best practice stacks multiple checks:

- Layer 1 - Transport Auth: Valid JWT, API key, or OAuth token?
- Layer 2 - Tenant Isolation: Does resource belong to user's org?
- Layer 3 - RBAC: Does user's role have the required permission?
- Layer 4 - Tool Policy: Is this specific tool on the allowlist?
- Layer 5 - Audit Log: Record who called what, when, and the result

Audit Logging

Every MCP tool invocation should be logged: who called it, which tool, which connection, success or failure, and latency. Do NOT log raw arguments if they may contain passwords or PII - log a hash instead.

Encryption at Rest

Sensitive data like MCP server passwords, OAuth tokens, and API secrets are encrypted before storing in the database using Fernet symmetric encryption. The encryption key lives in environment variables, not in the database.

Part 7: How Everything Fits Together

Outbound Flow: Orchestrator connects to external MCP

1. Admin creates MCP connection (base_url, auth type)
2. If OAuth: user completes login flow, tokens stored encrypted
3. Test connection: client calls tools/list, stores discovered tools
4. Builder binds specific tools to an agent config
5. At runtime: agent loop calls tools/call via McpHttpClient
6. Policy engine checks: RBAC + allowlist + audit log

Inbound Flow: External client uses Orchestrator MCP

1. External client (e.g. Cursor) reads /.well-known/oauth-protected-resource
2. User authenticates via OAuth (or uses API key for M2M)
3. Client calls tools/list on Orchestrator MCP server
4. Client calls tools/call (e.g. orchestrator_invoke_agent)
5. Orchestrator validates token, checks RBAC, executes internally

Glossary - Quick Reference

MCP	Model Context Protocol - standard for AI tool integration
Client	App that requests and calls tools
Server	System that exposes and runs tools
JSON-RPC	Message format for remote method calls
OAuth 2.1	Standard for delegated authorization
PKCE	Security extension preventing code interception
JWT	Signed token carrying user identity and role
API Key	Long-lived secret for machine access
RBAC	Permissions assigned via roles
Scope	Permission attached to an API key or OAuth token
Allowlist	Explicit list of permitted tools
Tenant	One organization's isolated data space
AS	Authorization Server - issues tokens
RS	Resource Server - accepts tokens (MCP server)
RFC 9728	MCP server discovery document standard
RFC 8707	Binds OAuth token to specific server URL